Spring AOP – Quick Revision Notes

1. What is Spring AOP

Spring AOP lets you run extra logic (before/after/around) on methods without touching the actual business code. Used for logging, security, auditing, etc.

2. Add AOP Dependency

Spring Boot 3.x → use `spring-boot-starter-aop`. Spring Boot 4.x → use `spring-boot-starter-aspectj`. No need to add `@EnableAspectJAutoProxy` manually.

3. Package Structure

Keep aspect classes inside the main app package (usually in an `aspect` folder) so Spring can auto-detect them.

4. Target Bean

The normal service class (like `StudentService`) whose methods you want to intercept. Its methods are called "target methods".

5. Aspect Class

A class marked `@Aspect` (says "this class has AOP logic") + `@Component` (registers it as a Spring bean). Both are needed.

6. @Before

Runs advice code before the pointcut is entered. Pointcut expression decides WHICH method(s) it applies to.

7. Pointcut Expression

Format: `execution(returnType package.Class.method(args))`. It tells Spring exactly which method to match.

8. Advice

The method containing the extra logic (like logging) that runs at a matched point. The matched real method is called the "target method".

9. Proxy Object

Spring puts a proxy in front of your real bean. Caller → Proxy → Advice → Target method.

You don't call the aspect directly — Spring does it automatically.

10. Target vs Proxy

Target = actual object with real business logic. Proxy = Spring-generated wrapper that intercepts calls. The bean you normally inject IS the proxy.

11. 5 Advice Types (Overview)

`@Before`, `@AfterReturning`, `@AfterThrowing`, `@After`, `@Around`.

12. try-catch-finally Mapping (Easiest way to remember)

- Before entering try → `@Before`
- After successful return → `@AfterReturning`
- Inside catch → `@AfterThrowing`
- Inside finally → `@After`
- Whole try-catch-finally block → `@Around`

13. @Before Advice

Runs before target method starts. Can read args (`joinPoint.getArgs()`) but CANNOT change them or skip the method. Can only stop execution by throwing an exception.

14. JoinPoint

Object that lets advice inspect the current method call: `getSignature()`, `getArgs()`, `getTarget()`, `getThis()`. Read-only — no control.

15. @AfterReturning

Runs only if method completes successfully (no exception escapes). Use `returning = "result"` to access the returned value. Cannot change the returned value (advice method should return void).

16. "Normal completion" meaning

If method internally catches its own exception and still returns something, that counts as normal completion → `@AfterReturning` runs, `@AfterThrowing` does NOT.

17. @AfterThrowing

Runs only when an exception escapes the method. Use `throwing = "exception"` to access it. Doesn't stop/handle the exception — it still propagates to caller. Can restrict by exception

type in the parameter.

18. @After

Like `finally` — always runs, whether method succeeds or fails. No access to result or exception. Good for cleanup (closing resources, clearing context, releasing locks).

19. Compare After types

- `@AfterReturning` → success only, has result
- `@AfterThrowing` → failure only, has exception
- `@After` → both cases, no data access

20. @Around – Most Powerful

Wraps the ENTIRE method call. Can: run code before/after, change arguments, change return value, catch/replace exceptions, skip the method, or call it multiple times.

21. ProceedingJoinPoint

Special version of JoinPoint used only in `@Around`. Has `proceed()` method to continue the call chain to the actual target method.

22. proceed()

Calling `joinPoint.proceed()` continues execution (through remaining aspects, then the real target method). If you don't call it, the target method never runs.

23. Must return proceed()'s result

Always do `Object result = joinPoint.proceed(); return result;` — otherwise caller gets `null` even though target method worked fine.

24. Why @Around returns Object

Because one generic aspect might apply to methods with different return types (String, int, boolean, void, etc.), so `Object` is used as a safe common type.

25. @Around Practical Uses

- Measure execution time (use try-finally around proceed())
- Modify arguments before proceeding: `joinPoint.proceed(modifiedArgs)`
- Modify/wrap the returned value
- Convert an exception into a fallback value (catch it inside advice)

- Retry logic — call `proceed()` more than once (risky for non-idempotent operations like payments/emails)

26. Danger of @Around

Since it's so powerful, misuse can hide business logic, swallow real errors, or cause duplicate operations (e.g., double payment on retry). Use only when really needed.

27. Which Advice to Use (Cheat Sheet)

Need	Use
Run logic before method	@Before
Only on success	@AfterReturning
Only on failure	@AfterThrowing
Always (cleanup)	@After
Change args/return value, skip, retry, timing	@Around

28. Golden Rule

Always use the narrowest/simplest advice that solves your need. Don't use `@Around` for everything just because it's powerful — it makes code harder to understand.

29. Full Flow (Mental Model)

Caller → Proxy → `@Around` (before proceed) → `@Before` → Target method → (success: `@AfterReturning` / failure: `@AfterThrowing`) → `@After` → back to `@Around` (after proceed) → Caller.